



Міністерство освіти і науки України
Національний технічний університет України «Київський політехнічний
інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №8
З дисципліни «Технології розроблення програмного забезпечення»
Тема: **«ШАБЛони «COMPOSITE», «FLYWEIGHT», «INTERPRETER»,
«VISITOR»»**
Музичний програвач

Виконав:
Студент групи ІА-23
Волошин В. В.

Перевірив:
Мягкий М. Ю.

Київ-2024

Зміст

Тема:.....	3
Мета:.....	3
Хід роботи.....	3
1. Реалізувати не менше 3-х класів відповідно до обраної теми.....	3
2. Реалізувати один з розглянутих шаблонів за обраною темою.....	4
Висновки:.....	6
Код проекту:.....	6

Тема:

ШАБЛОНИ «COMPOSITE», «FLYWEIGHT», «INTERPRETER», «VISITOR»

Мета:

Ознайомитися з основними шаблонами проектування, такими як «Composite», «Flyweight», «Interpreter», «Visitor» , дослідити їхні принципи роботи та навчитися використовувати для створення гнучкого та масштабованого програмного забезпечення, зокрема у розробці музичного програвача.

Хід роботи

1. Реалізувати не менше 3-х класів відповідно до обраної теми

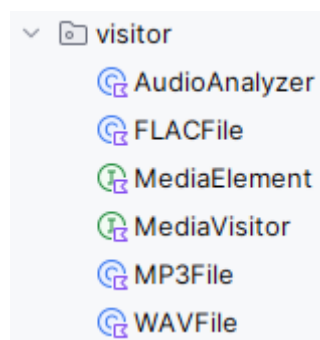


Рис. 1 — Структура проекту

У процесі виконання лабораторної роботи було реалізовано 6 класів, які забезпечують реалізацію функціоналу музичного програвача (рис. 1). Наведемо детальний опис кожного з цих класів:

1. MediaVisitor

Це інтерфейс, який визначає набір методів visit() для обробки різних типів аудіо-файлів (MP3, FLAC, WAV). Кожен метод відповідає конкретному типу файлу. Цей інтерфейс дозволяє реалізувати різноманітну логіку для кожного формату файлів, не змінюючи їхню основну структуру.

2. MediaElement

Це інтерфейс, який зобов'язує всі типи медіа-елементів (аудіо-файли) реалізовувати метод accept(visitor: MediaVisitor). Метод дозволяє медіа-елементу приймати відвідувача (Visitor) і передавати себе для обробки. Забезпечує єдиний інтерфейс для об'єктів, які можуть оброблятися відвідувачем, незалежно від їх типу.

3. MP3File

Клас, який представляє MP3-файли. Містить поля, такі як назва файлу (name) та бітрейт (bitrate). Реалізує метод accept(visitor: MediaVisitor),

який передає себе конкретному відвідувачу для обробки. Модель для роботи з MP3-файлами в програвачі. Дозволяє Visitor виконувати операції, специфічні для MP3-файлів, наприклад, аналіз бітрейту.

4. FLACFile

Клас, який представляє FLAC-файли. Містить поля, такі як назва файлу (name) та частота дискретизації (sampleRate). Так само реалізує метод accept(visitor: MediaVisitor) для прийняття відвідувача. Модель для роботи з FLAC-файлами. Дозволяє обробляти FLAC-файли окремо від інших типів файлів, враховуючи їх специфіку, наприклад, високу якість звуку.

5. WAVFile

Клас, який представляє WAV-файли. Містить поля, такі як назва файлу (name) та кількість каналів (channels). Реалізує метод accept(visitor: MediaVisitor). Модель для роботи з WAV-файлами. Забезпечує можливість обробки багатоканального аудіо або іншої специфіки формату WAV через Visitor.

6. AudioAnalyzer

Клас, що реалізує інтерфейс MediaVisitor. Містить реалізації методів visit() для кожного типу файлів (MP3, FLAC, WAV). У цих методах виконується конкретна логіка, наприклад, аналіз характеристик файлів (бітрейт, частота, кількість каналів). Це може бути використано, наприклад, для відображення детальної інформації про файл у програвачі або для перевірки якості аудіо.

2. Реалізувати один з розглянутих шаблонів за обраною темою

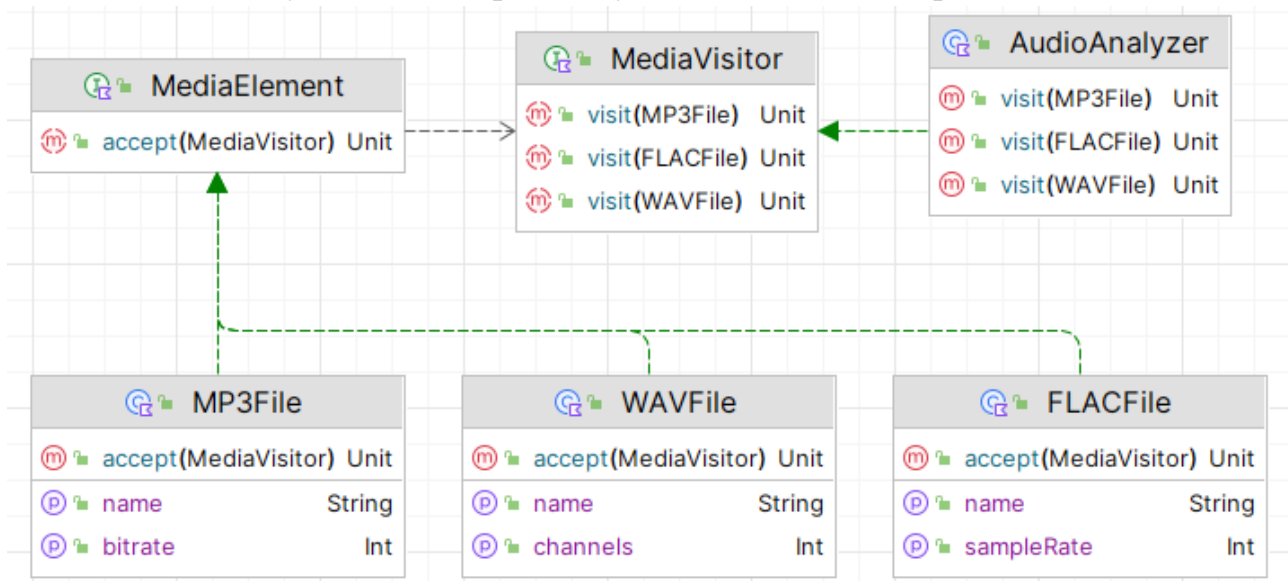


Рис. 2 — Діаграма класів

У проєкті музичного програвача патерн Visitor був реалізований для вирішення завдання обробки різних типів аудіо-файлів (MP3, FLAC, WAV) без зміни їхньої основної структури. Основна ідея полягала в тому, щоб розділити логіку, яка залежить від формату файлу, від самих класів файлів, зробивши її більш гнучкою та розширюваною.

Реалізація

- Елементи музичного програвача: Кожен тип аудіо-файлу (MP3, FLAC, WAV) реалізує інтерфейс MediaElement з методом accept(visitor: MediaVisitor). Цей метод дозволяє передати поточний об'єкт на обробку відвідувачу.
- Відвідувач: Відвідувач реалізує інтерфейс MediaVisitor, який визначає методи для кожного типу файлів (visit(mp3File: MP3File), visit(flacFile: FLACFile) і т.д.). Ці методи містять логіку, специфічну для кожного формату, наприклад, аналіз метаданих, перевірку якості або обробку.
- Взаємодія: Коли музичний програвач працює з плейлистами, він проходить через список медіафайлів і викликає метод accept() кожного елемента, передаючи відвідувача. Відвідувач, у свою чергу, обробляє кожен файл відповідно до його типу.

Проблеми, які вирішує патерн

- Розширюваність логіки обробки: Якщо з'явиться новий формат аудіо-файлу (наприклад, OGG), його легко інтегрувати, створивши новий клас-файл і додавши відповідний метод у Visitor. При цьому немає потреби змінювати вже існуючі класи.
- Розділення обов'язків: Логіка обробки медіа-файлів виноситься в окремі класи Visitor, що зменшує складність класів медіа-файлів і робить їх відповідальними лише за зберігання даних та забезпечення базового функціоналу.
- Єдина точка обробки: Вся специфічна логіка для роботи з аудіо-файлами зібрана у відвідувачах. Це спрощує підтримку та додачу нових функцій (наприклад, конвертація форматів, аналіз якості, експорт метаданих).

Переваги використання

- Гнучкість у розробці нових функцій: Можна додавати нових відвідувачів для реалізації різних сценаріїв роботи, наприклад, додавання функції конвертації форматів або візуалізації спектрограм.

- Зменшення зв'язності: Класи аудіо-файлів залишаються незмінними, навіть якщо логіка обробки суттєво змінюється або розширюється.
- Масштабованість: Завдяки Visitor нові формати або дії додаються швидко, не порушуючи існуючий код. Це забезпечує плавний розвиток додатку без ризику помилок у вже протестованих частинах.
- Чистота коду: Патерн допомагає уникнути великої кількості умовних конструкцій (if або when) у коді, зберігаючи його читабельність.

Висновки:

У ході лабораторної роботи було реалізовано патерн Visitor для обробки різних типів аудіо-файлів у додатку музичного програвача. Це дозволило спростити додавання нового функціоналу, розділити логіку обробки від структури файлів та забезпечити масштабованість і чистоту коду.

Код проекту:

<https://github.com/vv0loshyn/TRPZ-labs>